Petmor Sistema Veterinaria Spa

7. Refactorizacion aplicando Inversion de Control

Problema: dependencias creadas dentro de las clases

El enfoque inicial podia crear sus dependencias dentro del controlador:

```
// Antes: acoplamiento directo y dificil de sustituir en pruebas
var db = new ApplicationDbContext(opciones);
var cliente = new HttpClient();
```

La clase quedaba responsable de decidir como construir la base de datos y el

cliente HTTP.

Despues: dependencias inyectadas

El controlador recibe sus dependencias por constructor:

```
public class PetmorController(
    ApplicationDbContext db,
    AtencionService operaciones,
    EmisionVentaService emision) : Controller
{
    // El controlador usa servicios, pero no los crea.
}
```

ASP.NET Core registra y construye el grafo en `Program.cs`:

```
builder.Services.AddScoped<Petmor.Web.Services.AtencionService>();
builder.Services.AddScoped<Petmor.Web.Services.EmisionVentaService>();
builder.Services.AddHttpClient<Petmor.Web.Services.FactilizaService>();
```

```
flowchart TB
    P[Program.cs / DI Container] --> DB[ApplicationDbContext]
    P --> HTTP[HttpClient Factiliza]
    P --> AS[AtencionService]
    P --> VS[OperacionVentaService]
    P --> RS[ReservaService]
    C[Controllers] --> AS
    C --> VS
    C --> RS
```

Beneficios

- Menor acoplamiento.
- Sustitucion de dependencias en pruebas.
- Configuracion centralizada.
- Ciclo de vida controlado por ASP.NET Core.

```
flowchart TB
    P[Program.cs / contenedor DI] --> DB[ApplicationDbContext]
    P --> HTTP[HttpClient Factiliza]
    P --> AS[AtencionService]
    P --> VS[EmisionVentaService]
    C[Controllers] --> AS
    C --> VS
```